Understanding DQN with the TurtleBot3

A visual guide to the vanilla DQN experiment in `~/tb3_dqn`. Every number comes from `src/config.yaml`; every code pointer is a real file in the project.

1. The problem in one paragraph

A TurtleBot3 Burger sits in a 4 m x 4 m arena with 3 to 6 obstacles. A goal is placed somewhere. The robot must drive to the goal (closer than 0.2 m) without touching anything (a LiDAR reading below 0.13 m counts as a collision) within 500 steps. It decides what to do 10 times per second. It is never given a map: it only sees LiDAR distances and where the goal is relative to itself.

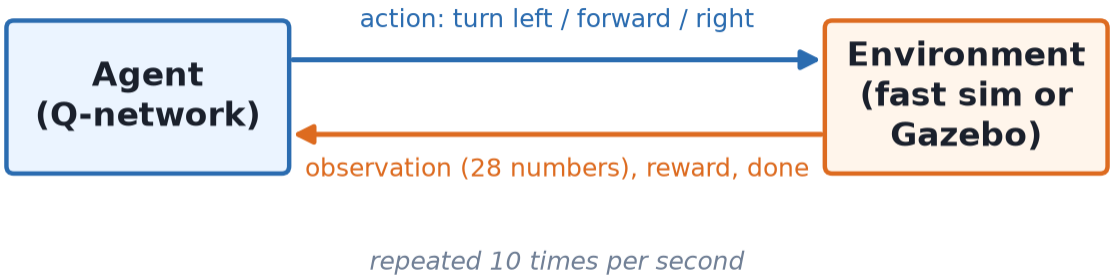


Figure 1. Reinforcement learning is a loop: the agent acts, the environment answers with a new observation and a reward.

The agent's job is to pick actions that maximise the **total discounted reward** over an episode.

2. State, action and reward

State: what the robot sees (28 numbers)

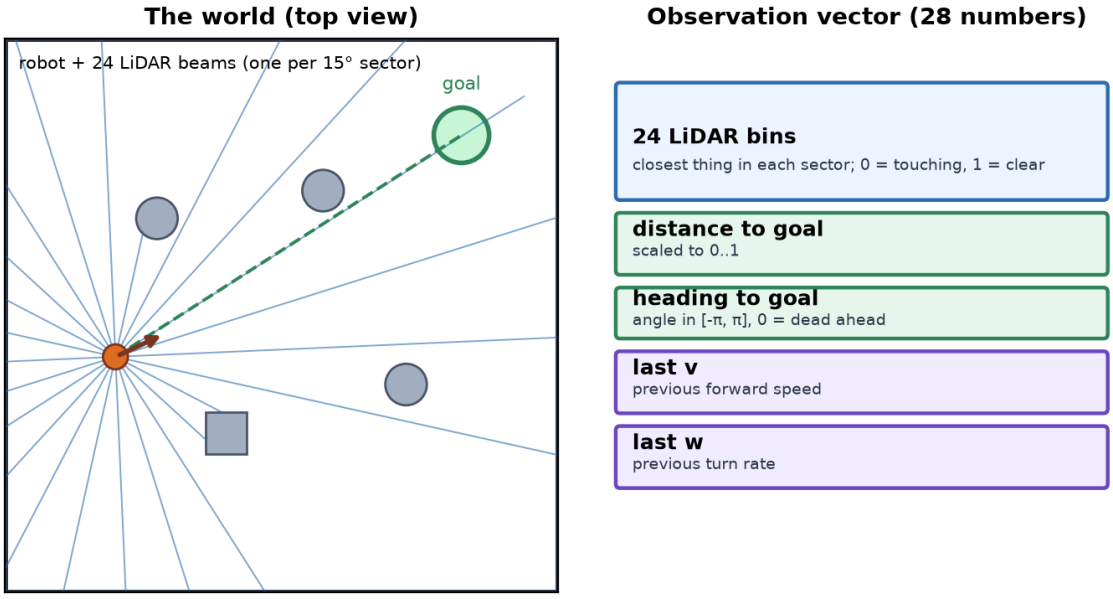


Figure 2. The 24 LiDAR beams are reduced to one number per 15° sector (the closest return), then joined by four numbers about the goal and the previous command.

The robot has no memory beyond "last v and last w", so it must decide from a single snapshot. (Older notes mention 29 dimensions; the code in `envs/common.py` gives 24 + 4 = 28.)

Actions: five discrete turns

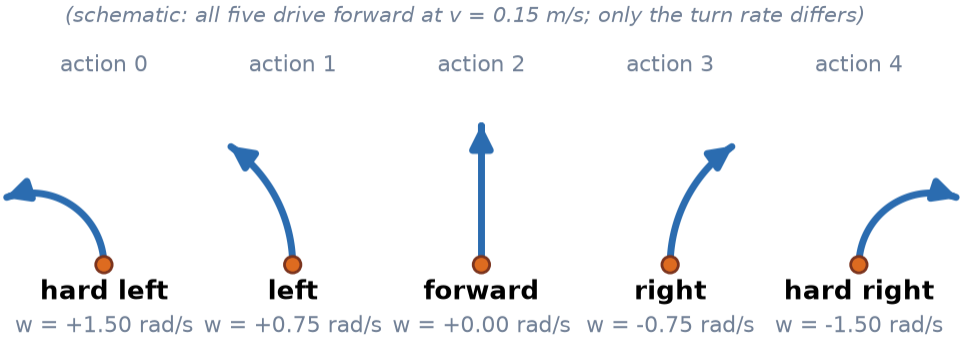


Figure 3. The robot always drives forward at 0.15 m/s. Only the turn rate changes. It can never stop or reverse.

"Discrete" is exactly why DQN works here: DQN needs a finite list of actions it can score. Phase 2 (TD3) will output a continuous (v, w) instead.

Reward: what "good" means

Event	Reward	Why
Reach goal	+100	the objective
Collide	-100	avoid obstacles
Every step	-0.05	nudges the robot to hurry
Progress	50 x (previous distance - current distance)	dense shaping signal, so learning does not wait for the episode's end

The progress term is potential-based, so driving in circles nets zero: the robot cannot farm it. Raising its weight from 10 to 50 was the one change that fixed the unstable first training run.

3. The core idea: the Q-function

Q(s, a) answers: "*if I am in state s, take action a now, and act well afterwards, how much total future reward do I expect?*" If you knew Q exactly, acting would be trivial: in each state, pick the action with the highest Q.

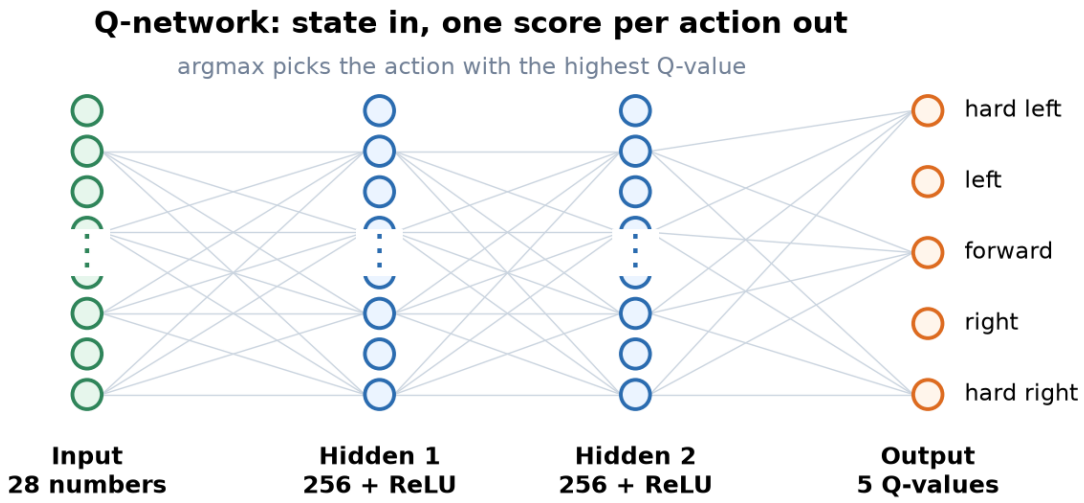


Figure 4. The network takes the 28 observation numbers and returns five scores, one per turn. (`QNetwork` in `agents/dqn.py`, two hidden layers of 256.)

Near an obstacle ahead-left, a good network gives high Q to "right" and low Q to "left". Future reward is discounted by gamma = 0.99: a reward 100 steps away counts as 0.99^{100}, about 0.37, of its face value. That is what lets the agent trade a short detour for a safe path.

4. How the network learns: the Bellman target

The true Q is unknown, so we make the estimate **self-consistent**. If the robot takes action a in state s, gets reward r and lands in s', then Q should satisfy:

```
Q(s, a)  ~=  r + gamma * max_a' Q(s', a')      (just r if the episode ended)
```

The right-hand side is the **target** y. Training shrinks the gap between the prediction Q(s, a) and y. The key lines in `DQNAgent.update`:

```
y    = rew * reward_scale + gamma * q_target(next_obs).max(dim=1).values * (1.0 - done)
loss = smooth_l1_loss(q_sa, y)
```

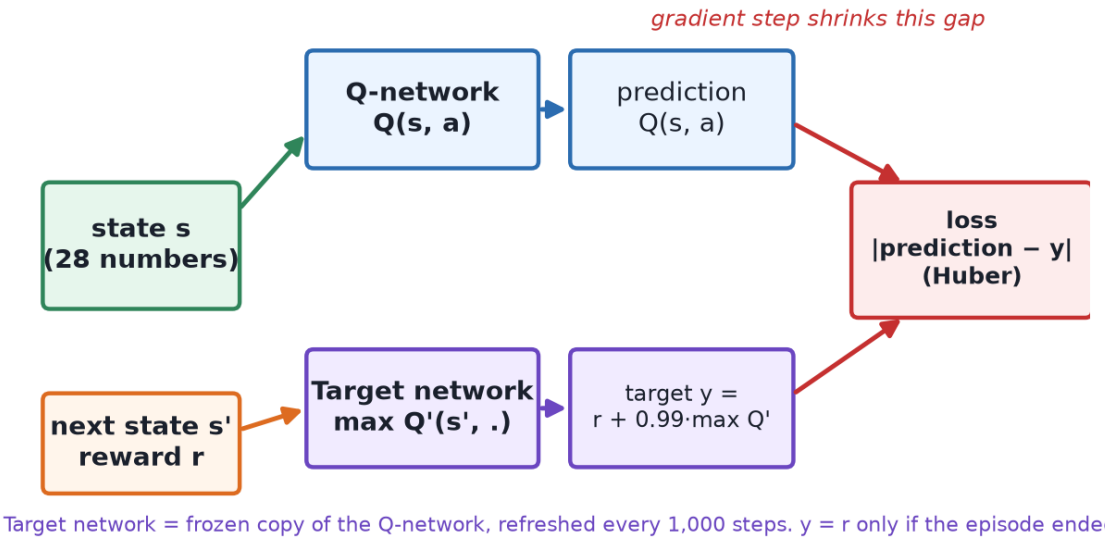


Figure 5. Prediction on one side, target on the other; the loss is the gap and each gradient step shrinks it.

- **(1 - done)**: if the robot just reached the goal or hit a wall there is no future, so the target is only r.
- **Vanilla DQN**: the same target network both picks the best next action and evaluates it. Double DQN splits those two jobs. This project deliberately uses neither Double DQN, Dueling nor prioritised replay.
- A 500-step timeout is not treated as terminal (the observation has no clock), so the agent still bootstraps.

5. The three tricks that make it work

Neural networks trained on a stream of correlated, moving-target data tend to blow up. DQN adds three stabilisers.

5.1 Replay buffer

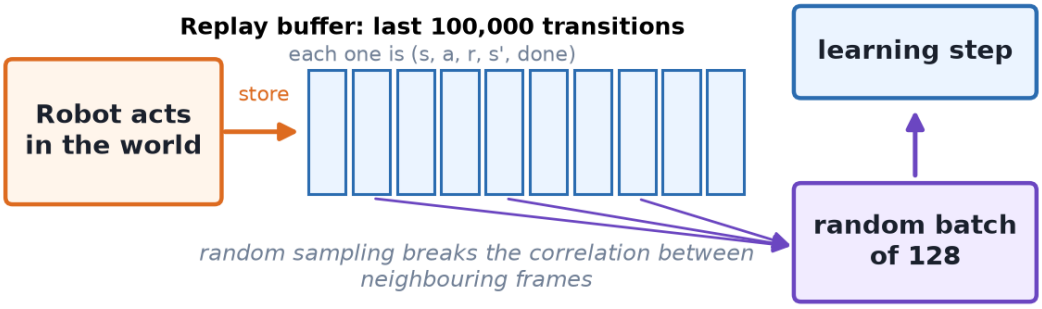


Figure 6. Every step is stored; learning samples a random batch of 128 from the last 100,000.

Consecutive frames are nearly identical (the robot barely moved). Training on them in order would overfit to the last few seconds. Random sampling breaks that correlation and reuses each experience many times.

5.2 Target network

A frozen copy of the Q-network used only to compute y, overwritten with the live weights every 1,000 steps (hard update). If the target moved at every gradient step, the network would be chasing its own tail.

5.3 Epsilon-greedy exploration

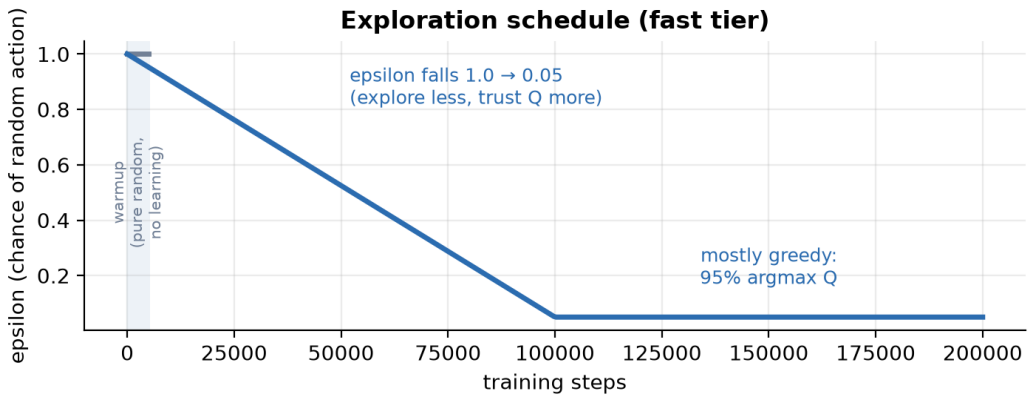


Figure 7. Epsilon is the probability of a random action. At evaluation time it is set to 0 (pure greedy).

The first 5,000 steps are pure random warmup so the buffer has something to learn from. Early on the robot flails and mostly crashes; as epsilon falls it increasingly trusts its Q-values. Extras: Huber loss (robust to the large ±100 terminal rewards) and gradient-norm clipping at 10.

6. One training step, end to end

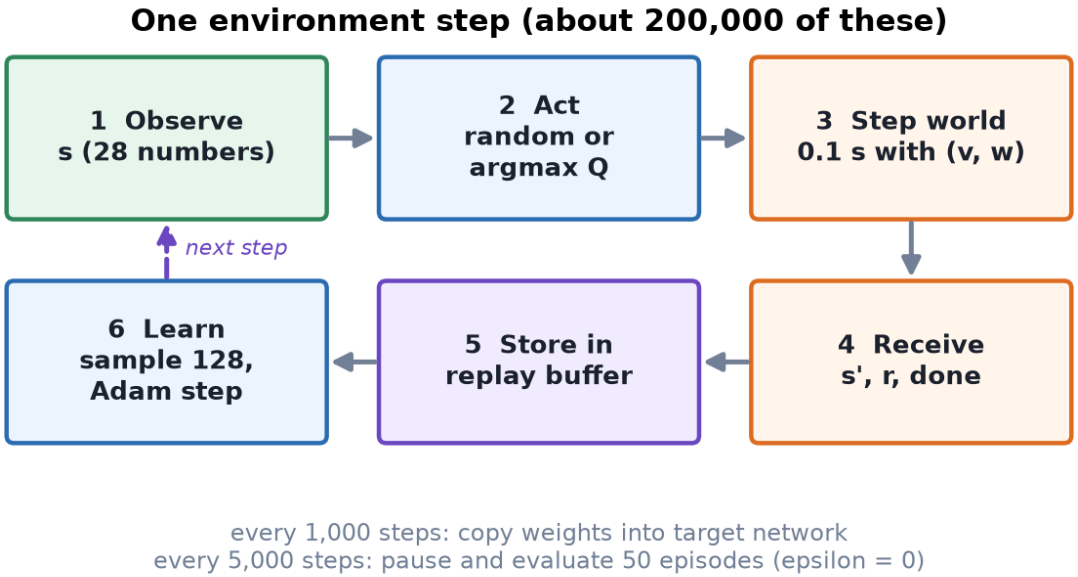


Figure 8. `train.py` *repeats this about 200,000 times on the fast tier.*

After warmup, steps 1 to 6 repeat every 0.1 s of simulated time. The Q-network goes from random garbage to a usable obstacle-avoiding navigator over the run.

7. Why two simulators?

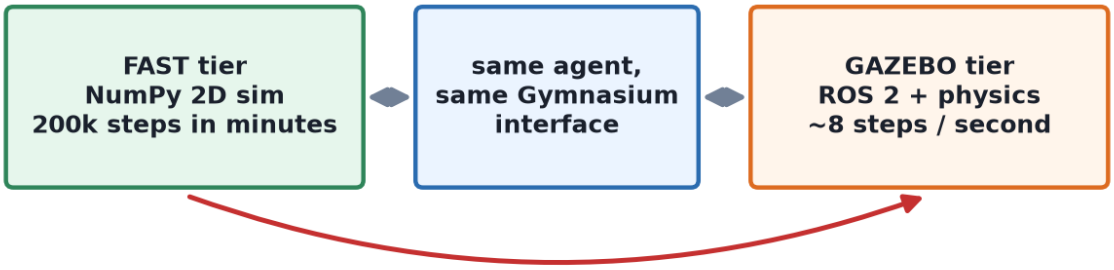


Figure 9. Both tiers expose the same Gymnasium interface, so the agent code never changes.

Gazebo runs near real time (about 7 to 10 steps/s), so 200k steps there would take most of a day. The plan is to train on the fast tier, then check the policy still works in Gazebo: **sim-to-sim transfer**.

8. What the results mean

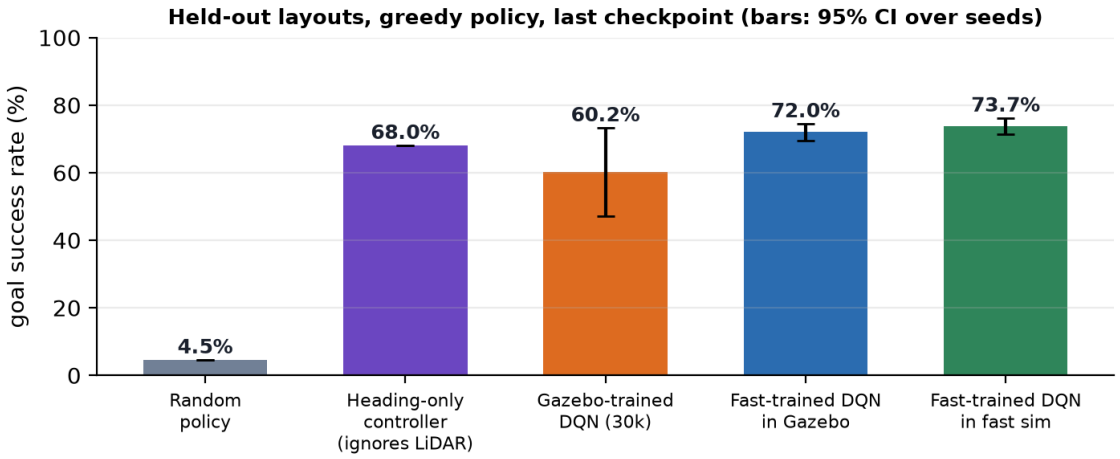


Figure 10. Greedy evaluation of the last checkpoint on 200 held-out layouts. Error bars: 95% CI over seeds (10 seeds, except Gazebo-trained which has 3).

Read this honestly:

- DQN went from about 4% to about 74%: it clearly learned.
- But a controller that ignores obstacles and just steers at the goal already scores 68%, so DQN's real gain over that is about 6 points. Many layouts are easy; the learned obstacle avoidance matters on the harder ones.
- Transfer to Gazebo works (gap 1.6 points) after two Gazebo defects were fixed: wheels losing traction after hard turns, and drifting wheel odometry being used as the pose.
- The Gazebo-trained bar is wide because it has only 3 seeds and 30k steps.

9. Common failure modes

Symptom	Cause	Where to look
Robot spins in place forever	Heading angle not wrapped to [-pi, pi]	envs/common.py
Learning stalls near 0%	Sparse reward, too little shaping	reward.progress_weight
Success collapses mid-training	Q-values diverge, unstable targets	results/q_stability.png
Circling or wall-hugging for reward	Reward hacking	progress term is potential-based
Great in fast sim, bad in Gazebo	Sim-to-sim gap: dynamics, contact, pose source	results/transfer_diag/report.md

10. Try it yourself

```
cd ~/tb3_dqn/src
../venv/bin/python train.py --agent dqn --env fast --smoke      # ~2k steps, sanity check
../venv/bin/python train.py --agent dqn --env fast --seed 0     # full run
../venv/bin/python evaluate.py --checkpoint ../results/runs/fast_dqn_seed0/final.pt --env fast
../venv/bin/python plot.py                                      # regenerate figures
```

Change one value in `src/config.yaml` at a time to build intuition:

- **reward.progress_weight**: 5 vs 50. How reward shaping changes learning speed.
- **dqn.target_update_every**: 100 vs 5000. Watch stability change.
- **dqn.gamma**: 0.9 vs 0.99. Shorter vs longer foresight.
- **tiers.fast.eps_decay_steps**: how long to explore.

11. Glossary

Term	Meaning
Episode	One attempt from reset to goal, collision or timeout.
Policy	The rule mapping state to action. Here: the action with the highest Q.
Bootstrapping	Using the network's own estimate of the future as part of the learning target.
Off-policy	DQN can learn from old, random transitions in the buffer, not only its current behaviour.
Greedy	Always pick the highest-Q action (epsilon = 0).
Sim-to-sim transfer	Train in one simulator, test in another.
Vanilla DQN	The 2015 recipe (replay + target network + epsilon-greedy) with no later improvements.

12. Where it goes next (phase 2)

DQN can only choose from 5 fixed turns. Phase 2 adds **TD3**, an actor-critic method that outputs a continuous (v, w), so the robot could slow down near obstacles or turn exactly as much as needed. The envs, logging and plotting were kept interface-stable so the comparison is like for like.